CRIPTA

Proyecto de Estructuras de Datos – II Ciclo 2026

Diseño, implementación y evaluación de estructuras de datos
en un juego de mazmorra dirigido por eventos

Aspecto	Definición
Modalidad	Grupos de hasta 4 personas. Pueden trabajar con personas en horarios diferentes (grupos 04 y 06)
Lenguaje de programación	Python 3.11 o superior
Porcentaje	15% de la nota final del curso
Uso de LLM	Permitido, con bitácora obligatoria de prompts (§10.3)

**Regla editorial del proyecto: las reglas del juego están especificadas;
las decisiones de estructuras de datos no.**

Antes de comenzar: acceso al API

El servicio HTTP del proyecto ya está publicado. Antes de implementar la lógica del juego, verifiquen que su programa puede consultar el API desde Python y procesar una respuesta JSON.

Recurso	Dirección
Base de los endpoints	https://cripta-api.kad06a0zhgs84.us-east-2.cs.amazonlightsail.com/v1
Documentación interactiva (Swagger)	https://cripta-api.kad06a0zhgs84.us-east-2.cs.amazonlightsail.com/docs

Encabezado obligatorio: `X-Cripta-Client-Id`

> Todas las solicitudes HTTP al servicio deben incluir el encabezado `X-Cripta-Client-Id`. El valor debe generarse una sola vez al iniciar cada ejecución del programa y reutilizarse en todas las solicitudes de esa ejecución. No generen un UUID nuevo para cada solicitud. Al iniciar una nueva ejecución del programa, pueden generar un identificador diferente.

Ejemplo mínimo en Python

```
import uuid
import requests

BASE_URL = "https://cripta-api.kad06a0zhgs84.us-east-2.cs.amazonlightsail.com/v1"
client_id = str(uuid.uuid4())
headers = {"X-Cripta-Client-Id": client_id}

respuesta = requests.get(
    f"{BASE_URL}/criptas",
    headers=headers,
    timeout=10,
)

print(respuesta.status_code)
print(respuesta.json())
```

Información práctica para empezar

- El mismo identificador se usa durante toda la ejecución. Inclúyanlo al listar criptas, consultar datos generales, descargar páginas, contenido, catálogo, versiones y también en reintentos.
- La documentación Swagger es la referencia para el contrato HTTP exacto (rutas, parámetros y forma de las respuestas). Este enunciado sigue siendo la referencia para las reglas del juego, restricciones de estructuras de datos, rendimiento y evaluación.
- HTTP 429 debe manejarse sin que el programa se caiga. El tiempo real de espera nunca avanza el reloj virtual. Además, un reintento cuenta dentro del presupuesto de solicitudes definido por la cripta.
- Aprovechen las consultas por lotes. El contenido acepta hasta 10 salas y el catálogo hasta 10 identificadores por solicitud, tal como se detalla en §3.3 y §3.4.
- La biblioteca `requests` está permitida. Usen un tiempo de espera explícito y reporten errores con contexto, en lugar de dejar que una excepción de red termine el programa sin explicación.

1. Propósito del proyecto

Este proyecto no se trata de programar un juego por el juego mismo. Se trata de tomar decisiones de diseño bajo restricciones reales, implementar las estructuras de datos necesarias y defender esas decisiones con evidencia.

Cripta es el escenario sobre el cual se plantean problemas de selección, organización, búsqueda, actualización, persistencia y ordenamiento de datos. El juego es suficientemente pequeño para poder comprenderlo por completo, pero suficientemente dinámico para que una decisión inadecuada tenga consecuencias medibles.

En este enunciado no se les dice qué estructura de datos usar para cada problema. Se describen las operaciones que el programa debe ejecutar y las propiedades que deben cumplir. Elegir una estructura, implementarla y justificar por qué es apropiada para su caso es parte central del proyecto.

Importante. Un programa que "funciona" pero cuyas decisiones de estructuras de datos no están justificadas con mediciones propias no satisface el objetivo del proyecto. Del mismo modo, una buena explicación no sustituye una implementación correcta.

El contenido de las criptas no debe estar escrito en el programa. El profesor proveerá un servicio web con varias criptas publicadas, incluyendo criptas pequeñas a partir de aproximadamente 20 salas y criptas considerablemente mayores. En la revisión se utilizará una cripta no publicada previamente.

2. Reglas completas de Cripta

Esta sección define las reglas funcionales del juego. Estas reglas no son sugerencias de implementación: todos los programas deben producir el mismo resultado observable ante la misma cripta, la misma semilla aleatoria y la misma secuencia de acciones del jugador. Eso sí, no se exige que programas de grupos distintos coincidan, ya que eso depende de la lógica que cada grupo implementó.

2.1 La cripta y sus salas

Una cripta es un conjunto de salas conectadas por salidas en cuatro direcciones: N (norte), S (sur), E (este) y O (oeste). Una salida puede no existir, estar abierta o estar cerrada con una llave. Cada salida conecta exactamente dos salas.

El jugador y los enemigos ocupan una sala completa; no existe una cuadrícula ni posiciones dentro de una sala. Todos los actores que se encuentran en la misma sala se consideran adyacentes entre sí.

Cada cripta declara una sala inicial, una sala de salida y la llave necesaria para abrir la puerta que permite llegar a la salida. La cripta también declara las estadísticas iniciales del jugador, la capacidad de su inventario y el presupuesto máximo de solicitudes de red para esa partida.

2.2 Actores y estadísticas

Un actor es el jugador o una instancia concreta de enemigo. Dos enemigos del mismo tipo comparten la misma ficha de catálogo, pero son actores distintos: cada uno tiene su propia vida, sala, identificador de instancia y tiempo de próxima acción.

Atributo	Significado
vida / vida_max	Al llegar a 0, el actor muere. vida nunca puede superar vida_max.
ataque	Valor base usado para calcular el daño que inflige.

Atributo	Significado
defensa	Valor que se resta al daño recibido.
velocidad	Determina cuánto tiempo virtual tarda en completarse una acción. La referencia es 100.
tiempo_siguiente	Instante del reloj virtual en el que el actor podrá ejecutar su próxima acción.
comportamiento	Solo enemigos: guardián, errante o rastreador (§2.8).

Las estadísticas iniciales del jugador no están quemadas en el programa: vienen en los datos generales de la cripta (§3.3). Los enemigos obtienen sus estadísticas base del catálogo. Una instancia de enemigo puede indicar una vida inicial menor que vida_max.

2.3 Reloj virtual, eventos y orden de ejecución

El juego no avanza por rondas. Mantiene un reloj virtual entero, medido en unidades de tiempo. Actores y efectos temporales se representan como eventos programados.

repetir mientras la partida siga activa:
1. obtener el evento pendiente con menor tiempo_siguiente
2. avanzar el reloj virtual a ese instante
3. ejecutar el evento
4. si corresponde, programar su siguiente ocurrencia

Cuando dos eventos tienen el mismo tiempo_siguiente, se desempatan por un número de secuencia creciente asignado al programarlos. Menor tiempo y luego menor número de secuencia define un orden total y determinista.

El número de secuencia es parte de la semántica del juego, no una medición de tiempo real. Nunca se usa la hora del sistema ni el orden de llegada de respuestas HTTP para decidir qué evento ocurre primero.

Costo de una acción

Acción	Costo base
Moverse a una sala vecina	100
Atacar	100
Esperar	100
Usar un objeto	50
Equipar un objeto	50
Recoger un objeto del suelo	25
Soltar un objeto	25

Acción	Costo base
Abrir una puerta con llave	50

Después de que un actor ejecuta una acción de costo c, su próxima acción se programa con el intervalo:

```
intervalo = max(1, costo * 100 // velocidad)
tiempo_siguiente = reloj_actual + intervalo
```

Ejemplo: con una acción de costo 100, un actor de velocidad 100 vuelve a actuar 100 unidades después; uno de velocidad 150 vuelve a actuar 66 unidades después. Por eso un enemigo rápido puede actuar más veces que el jugador entre dos decisiones del jugador.

Cambio de velocidad mientras una acción está programada

Si un efecto cambia la velocidad de un actor que ya tiene una próxima acción programada, el tiempo restante se escala para conservar el progreso ya realizado. Si el cambio ocurre en el instante t:

```
restante_nuevo = max(1, (tiempo_siguiente - t) * velocidad_anterior // velocidad_nueva)
tiempo_siguiente = t + restante_nuevo
```

Esto significa que una aceleración acerca la próxima acción y una ralentización la aleja. La operación debe mantener el desempate determinista de eventos. Eso sí, toda reprogramación asigna un número de secuencia nuevo.

2.4 Inicio de una partida y activación de enemigos

La partida empieza con reloj virtual 0. El jugador se encuentra en la sala inicial y su primera acción está programada para tiempo 0.

El contenido de la sala inicial debe cargarse antes de aceptar la primera acción. El programa también puede precargar otras salas de acuerdo con su estrategia de red (§4.2).

Incorporar el contenido de una sala al modelo no activa a sus enemigos. Las instancias de enemigos de una sala pasan a estar activas cuando el jugador entra a esa sala. El jugador se considera dentro de la sala inicial desde el instante 0, de modo que los enemigos de la sala inicial se activan en ese momento.

Un enemigo ya activo que se desplaza a otra sala permanece activo, sin importar si el jugador ha entrado alguna vez a esa sala. Una sala puede contener simultáneamente enemigos dormidos —los que trae su contenido y el jugador aún no ha visitado— y enemigos activos que llegaron desde otra sala.

La primera acción de un enemigo recién activado se programa a partir del reloj virtual actual usando una acción base de costo 100. Por ejemplo, un enemigo de velocidad 150 activado en tiempo 0 tendrá tiempo_siguiente = 66. Cuando varios enemigos se activan en el mismo instante, se programan en orden ascendente de su id de instancia, de modo que reciban números de secuencia deterministas.

Determinismo de red. La latencia real del servicio no puede cambiar la simulación. Las respuestas pendientes se incorporan al modelo en un orden determinista establecido por el programa (por ejemplo, el orden de solicitud). Esperar una respuesta de red nunca avanza el reloj virtual. Si una acción intenta entrar a una sala cuyo contenido solicitado aún no llegó, la acción no se confirma hasta disponer de ese contenido.

2.5 Movimiento, entrada a una sala y puertas

Moverse es una acción normal de costo 100. Para moverse debe existir una salida en la dirección indicada y la puerta correspondiente debe estar abierta. El jugador aparece en la sala destino al ejecutar la acción; no existe un ataque gratuito por entrar o salir.

Al entrar a una sala se muestra inmediatamente la información que el jugador puede observar. Por ello, antes de confirmar el movimiento, el contenido de la sala destino debe estar disponible en memoria (§4.2).

Después del movimiento, el jugador no vuelve a actuar hasta su nuevo tiempo_siguiente. Durante ese intervalo se ejecutan normalmente todos los eventos cuyo tiempo sea menor. Por ejemplo, si después de entrar a una sala el jugador queda programado para tiempo 300 y un enemigo de esa sala está programado para tiempo 250, el enemigo actuará primero y atacará al jugador en tiempo 250.

Abrir una puerta y atravesarla son dos acciones distintas. Si el jugador posee la llave requerida, puede abrir la puerta con una acción de costo 50. La llave no se consume. Una puerta abierta permanece abierta, salvo que la salida declare un cierre automático; en ese caso se programa un evento de cierre después del intervalo indicado por la cripta.

Los enemigos no poseen inventario, no recogen llaves y no pueden abrir puertas cerradas. Solo pueden atravesar salidas abiertas.

2.6 Combate

Si un actor decide atacar, debe encontrarse en la misma sala que su objetivo. Los enemigos solo atacan al jugador. El jugador puede seleccionar cuál enemigo atacar cuando hay varios en la sala.

daño = max(1, ataque_atacante + azar.randint(0, 4) - defensa_defensor)

donde azar es la instancia random.Random(semilla) del modelo. randint(0, 4) incluye ambos extremos: produce 0,1,2,3 o 4

El daño mínimo siempre es 1. Cuando la vida de un actor llega a 0, muere inmediatamente y deja de ejecutar eventos futuros. Si un enemigo tiene objetos en su campo suelta, esos objetos se colocan en el suelo de la sala al morir.

El jugador puede abandonar una sala aunque haya enemigos vivos. No existen ataques de oportunidad ni una fase especial de huida.

Cada instancia de enemigo debe ser identificable en la interfaz. Una presentación válida sería “Rata gigante #1”, “Rata gigante #2”, aunque la forma exacta de mostrarla queda a decisión del grupo.

Azar reproducible

Toda aleatoriedad del modelo utiliza una instancia propia random.Random(semilla). No se permite depender del generador global del módulo random. La semilla se fija al comenzar la partida y se guarda tanto en el archivo de partida como en la bitácora de reproducción.

Dos ejecuciones con la misma versión de cripta, la misma semilla y la misma secuencia de acciones del jugador deben producir la misma partida. Esto incluye la misma secuencia de llamadas al generador aleatorio.

2.7 Inventario, equipo y objetos

La capacidad máxima del inventario viene en los datos generales de la cripta como inventario_max. Cada objeto ocupa una posición, sin importar su peso. Si el inventario está lleno, el jugador no puede recoger otro objeto hasta soltar alguno.

Recoger y soltar afectan un objeto por acción. Soltar un objeto lo coloca en el suelo de la sala actual. Equipar un objeto lo mueve al frente del orden real del inventario y aplica su efecto de equipo. El jugador puede tener equipada como máximo un arma y una armadura; equipar otra del mismo tipo sustituye a la anterior, que permanece en el inventario.

El peso y el valor se usan para vistas y ordenamientos del inventario; no existe un límite de peso total.

Clases de objetos del catálogo

clase	Campos relevantes	Regla
arma	ataque_bonus, peso, valor	Mientras está equipada, suma ataque_bonus al ataque del jugador.
armadura	defensa_bonus, peso, valor	Mientras está equipada, suma defensa_bonus a la defensa del jugador.
pocion	cura o modificador_velocidad, duracion, peso, valor	Se consume al usarla. Puede curar o aplicar un efecto temporal de velocidad.
antidoto	peso, valor	Se consume al usarlo y cancela el veneno activo del jugador.
llave	abre, peso, valor	Permite abrir puertas cuya llave coincide con su id. No se consume.
antorcha	duracion, peso, valor	Se consume una unidad de uso al encenderla y permanece encendida durante duracion unidades virtuales.
pergamino_retroceso	peso, valor	Se consume al usarlo y revierte la última acción del jugador según §2.11.

Una cripta puede contener varias instancias del mismo tipo de objeto. La ficha técnica se comparte; la pertenencia al inventario o al suelo corresponde a la instancia.

2.8 Comportamiento de los enemigos

Cuando llega el turno de un enemigo, primero verifica si comparte sala con el jugador. Si es así, ataca y no ejecuta movimiento. Si no comparte sala, aplica su comportamiento:

Comportamiento	Acción cuando el jugador no está en su sala
Guardián	Espera. Nunca abandona voluntariamente su sala.
Errante	Escoge al azar una de las salidas abiertas y transitables. Si no hay ninguna, espera.
Rastreador	Examina únicamente las salas vecinas accesibles y sigue el rastro fresco más reciente (§2.9). Si no encuentra uno, espera.

Atacar, moverse o esperar son acciones de costo 100. Por tanto, el tiempo de la siguiente acción del enemigo se calcula de la misma forma en los tres casos.

2.9 El rastro del jugador

Cada sala registra el instante del reloj virtual en el que el jugador estuvo allí por última vez. Al iniciar la partida, la sala inicial recibe rastro en tiempo 0. Al completar un movimiento, la sala destino recibe el tiempo actual.

En su turno, un rastreador examina únicamente las salas vecinas conectadas por salidas abiertas. Se mueve hacia la que tenga el rastro más fresco siempre que ese rastro tenga menos de 400 unidades de antigüedad.

Si dos vecinas tienen exactamente el mismo tiempo de rastro, se escoge la sala con menor id. Si ninguna vecina tiene rastro fresco, el enemigo espera. El rastreador no conoce el mapa completo y no realiza búsqueda de rutas.

2.10 Efectos temporales y del entorno

Además de las acciones de actores, el reloj puede contener eventos de efectos. Los siguientes efectos forman parte del contrato del juego:

Efecto	Regla
Veneno	Aplica el daño indicado cada 80 unidades. Dura 400 unidades desde su aplicación, salvo que un antídoto lo cancele antes.
Cambio de velocidad	Modifica temporalmente la velocidad. Al comenzar y terminar el efecto debe reajustarse la próxima acción según §2.3.
Antorcha	Al encenderse programa su apagado después de la duración indicada por el objeto.
Regeneración	Un enemigo que declare regeneración recupera la cantidad indicada cada 200 unidades, sin superar vida_max.
Trampa	Al activarse aplica su efecto, queda desarmada y programa su rearme 300 unidades después, salvo que la ficha indique otro intervalo.
Cierre de puerta	Una salida con cierre automático programa el cierre después del tiempo indicado al abrirse.

Las trampas se describen en el contenido de una sala. Una trampa armada se activa cuando el jugador entra a la sala, inmediatamente después de completar el movimiento y antes de procesar el siguiente evento del reloj. Si está desarmada, no ocurre nada hasta su rearme.

Cancelar un efecto impide sus activaciones futuras. El diseño de §4.1 debe contemplar tanto la muerte de actores como la cancelación de efectos sin depender de recorrer todos los eventos pendientes.

2.11 Pergamino de retroceso

Un pergamino de retroceso revierte el estado del juego hasta el punto inmediatamente anterior a la última acción ejecutada por el jugador. No es un movimiento hacia la sala anterior: es una operación de deshacer sobre el estado completo. La acción de utilizar un pergamino de retroceso no consume tiempo virtual y no se registra como acción del jugador en el historial.

La reversión incluye la propia acción del jugador y todo lo que haya ocurrido después de ella antes de la siguiente acción del jugador: acciones enemigas, efectos, reloj, eventos programados o cancelados, rastro, cambios de vida, inventario, objetos del suelo, estado de puertas y trampas, y cualquier otro dato modificado.

Solo pueden deshacerse tantas acciones como pergaminos se tengan y nunca más de 5 acciones del jugador hacia atrás. El pergamino utilizado se descuenta del inventario al usarlo, y ese descuento queda fuera del alcance de la reversión: la reversión no restaura pergaminos de retroceso ni modifica su cantidad. Sin esta excepción, restaurar el inventario devolvería el pergamino consumido y una sola unidad permitiría retroceder indefinidamente.

La excepción aplica en ambos sentidos. Si durante el intervalo revertido el jugador recogió o soltó un pergamino, la reversión tampoco deshace ese cambio: la cantidad de pergaminos queda tal como estaba al momento de invocar el retroceso, menos la unidad consumida.

Importante. Está prohibido implementar el retroceso guardando copias completas del estado de la partida. Debe registrarse únicamente la información necesaria para invertir los cambios de cada intervalo entre decisiones del jugador.

El estado de la caché no debe estar dentro del alcance del retroceso ya que es un estado de la infraestructura, no es un estado del juego.

Ejemplo de retroceso

```
t=100  Jugador ataca a Rata #1. Rata pierde 7 de vida.
t=120  Evento de veneno: jugador pierde 2 de vida.
t=150  Rata #1 ataca: jugador pierde 4 de vida.
t=180  Se rearma una trampa.
t=200  Llega la siguiente acción del jugador.

Si en t=200 el jugador usa un pergamino, el estado vuelve al instante
inmediatamente anterior al ataque de t=100: se restaura la vida de la rata,
la vida del jugador, el estado de la trampa, el reloj, los eventos pendientes
y el rastro correspondientes. El pergamino queda consumido.
```

2.12 Victoria, derrota y puntaje

- Derrota: la vida del jugador llega a 0.
- Victoria: el jugador entra a la sala de salida después de haber abierto con la llave correspondiente la puerta que permite acceder a ella.

Al terminar, la partida se registra en una tabla de puntajes local con: nombre del jugador, id y versión de la cripta, acciones ejecutadas por el jugador, enemigos derrotados y tiempo final del reloj virtual.

En este documento, “acción del jugador” es una acción que consume tiempo virtual del jugador. No se usa “turno” como sinónimo de ronda, porque el juego no tiene rondas.

2.13 Ejemplo de una secuencia de juego

El siguiente ejemplo ilustra la interacción entre reloj, velocidad, movimiento y eventos. Los números son únicamente ilustrativos; las estadísticas reales vienen de la cripta y del catálogo.

```
Inicio: reloj = 0
Jugador: velocidad 100, tiempo_siguiente = 0
Rata gigante: velocidad 150, activada al entrar el jugador a su sala,
        tiempo_siguiente = 66

0   Jugador se mueve al norte (costo 100).
    Su próxima acción queda en 100. La sala norte ya estaba cargada.

66  La rata actúa. Si ahora comparte sala con el jugador, ataca.
    Su próxima acción queda en 132.

100 El jugador recoge una daga (costo 25).
    Su próxima acción queda en 125.

125 El jugador equipa la daga (costo 50).
    Su próxima acción queda en 175.

132 La rata vuelve a actuar antes que el jugador y ataca.

175 El jugador vuelve a tener una decisión.
```

Obsérvese que el orden surge únicamente de tiempo_siguiente. Entrar a una sala no concede ataques gratuitos ni reinicia los tiempos de los enemigos.

3. El servicio web

El curso proveerá un servicio HTTP de solo lectura. El objetivo no es aprender desarrollo web: basta con realizar solicitudes GET, interpretar códigos básicos y procesar JSON.

3.1 Reglas mínimas de HTTP

- respuesta.status_code == 200: la solicitud tuvo éxito; respuesta.json() devuelve el cuerpo convertido a estructuras de Python.
- respuesta.status_code == 429: el cliente excedió temporalmente la frecuencia permitida. El cuerpo incluye {"error": "...", "reintentar_en": 12}. Debe esperarse el tiempo indicado y reintentar sin que el programa se caiga.
- Cualquier otro código debe reportarse con un mensaje comprensible que incluya la operación que falló. No se acepta terminar con una excepción de requests sin contexto.

El tiempo de espera de red es tiempo real y no modifica el reloj virtual del juego.

3.2 Esqueleto, contenido y catálogo

El servicio separa la información para obligar al programa a resolver referencias y decidir qué conviene descargar y conservar:

- Esqueleto: salas y conexiones. Es liviano y se descarga completo por páginas al iniciar una cripta.
- Contenido: enemigos, objetos y trampas presentes en cada sala. Se solicita por lotes para las salas necesarias.
- Catálogo: fichas técnicas reutilizables de enemigos, objetos y trampas. Un tipo descargado puede aparecer muchas veces e incluso en criptas distintas.

Una instancia no es una ficha. Por ejemplo, dos ratas gigantes en una sala son dos actores distintos pero hacen referencia al mismo id de catálogo ent_rata_gigante.

3.3 Consultas disponibles

Base de los ejemplos: https://cripta-api.kad06a0zhgs84.us-east-2.cs.amazonlightsail.com/v1
Documentación interactiva (Swagger): https://cripta-api.kad06a0zhgs84.us-east-2.cs.amazonlightsail.com/docs

Listar las criptas publicadas

```
GET /criptas

{ "criptas": [
 { "id": "cripta-01", "nombre": "Osario menor", "salas": 20, "dificultad": 1 },
 { "id": "cripta-02", "nombre": "Bóvedas negras", "salas": 60, "dificultad": 2 }
]}
```

Datos generales de una cripta

```
GET /criptas/cripta-01

{
 "id": "cripta-01",
 "version": "a3f91c",
 "salas_total": 20,
 "paginas": 1,
 "salas_por_pagina": 50,
 "sala_inicial": 1,
 "sala_salida": 19,
 "llave_salida": "itm_llave_negra",
 "presupuesto_solicitudes": 9,
 "inventario_max": 10,
 "jugador": {
  "vida_max": 30,
  "ataque": 6,
  "defensa": 2,
  "velocidad": 100
 }
}
```

presupuesto_solicitudes es el máximo de solicitudes HTTP de red permitidas para esa partida. El valor lo define cada cripta y no debe calcularse ni quemarse en el programa. Una cripta de 20 salas puede tener un presupuesto distinto de una de 200 salas.

Esqueleto, por páginas de hasta 50 salas

```
GET /criptas/cripta-01/salas?pagina=1

{
 "pagina": 1,
 "total_paginas": 1,
 "salas": [
  { "id": 1, "nombre": "Entrada derruida",
    "salidas": { "N": { "sala": 2 } } },
  { "id": 2, "nombre": "Pasillo de las urnas",
    "salidas": {
     "S": { "sala": 1 },
     "N": { "sala": 3 },
     "E": { "sala": 7, "cerrada": true,
          "llave": "itm_llave_bronce", "cierre_automatico": 500 }
    }
  }
 ]
}
```

Contenido de salas, hasta 10 salas por consulta

```
GET /criptas/cripta-01/contenido?salas=1,2,3

{
 "contenido": [
  { "sala": 1, "enemigos": [],
    "objetos": ["itm_antorcha"], "trampas": [] },
  { "sala": 2,
    "enemigos": [
     { "instancia": "e-201", "tipo": "ent_rata_gigante", "vida": 12 },
     { "instancia": "e-202", "tipo": "ent_rata_gigante", "vida": 12 }
    ],
    "objetos": ["itm_daga_oxidada"],
    "trampas": [{ "instancia": "t-17", "tipo": "trp_dardos" }] }
 ]
}
```

Los ids de instancia son únicos dentro de la cripta y permiten distinguir enemigos o trampas concretos. El campo tipo apunta al catálogo.

Catálogo por lotes, hasta 10 identificadores

```
GET /catalogo?ids=ent_rata_gigante,itm_daga_oxidada,itm_antorcha,trp_dardos

{
 "entidades": [
  { "id": "ent_rata_gigante", "clase": "enemigo", "nombre": "Rata gigante",
    "vida_max": 12, "ataque": 5, "defensa": 1, "velocidad": 150,
    "comportamiento": "rastreador", "suelta": ["itm_pocion_menor"] },

  { "id": "itm_daga_oxidada", "clase": "arma", "nombre": "Daga oxidada",
    "peso": 2, "valor": 15, "ataque_bonus": 3 },

  { "id": "itm_antorcha", "clase": "antorcha", "nombre": "Antorcha",
    "peso": 1, "valor": 5, "duracion": 2000 },

  { "id": "trp_dardos", "clase": "trampa", "nombre": "Dardos en la pared",
    "daño": 4, "rearme": 300 }
 ]
}
```

También existe GET /catalogo/<id>, que devuelve una sola ficha, pero tarda aproximadamente lo mismo que solicitar un lote completo. Su uso repetido normalmente desperdicia presupuesto.

Versiones

```
GET /criptas/cripta-01/version
{ "id": "cripta-01", "version": "a3f91c" }

GET /catalogo/version
{ "version": "cat-2026-08-01" }
```

La versión de la cripta permite validar esqueleto y contenido persistidos. La versión global del catálogo permite saber si las fichas guardadas siguen vigentes sin descargarlas nuevamente.

3.4 Características del servicio que condicionan el diseño

- Cada solicitud tarda alrededor de 400 ms, independientemente de si devuelve uno o diez elementos.
- Contenido acepta hasta 10 salas por solicitud.
- Catálogo acepta hasta 10 ids por solicitud.
- Existe un límite de frecuencia; excederlo produce HTTP 429.

- El catálogo se comparte entre criptas.
- El presupuesto_solicitudes pertenece a la cripta y debe respetarse en cada partida.
- Una respuesta almacenada localmente no cuenta como solicitud de red.

Modo sin conexión. El curso publicará un paquete de respuestas previamente descargadas. La bandera --offline debe hacer que el programa lea esas respuestas desde disco a través del mismo contrato lógico que usa el cliente de red. El modelo no debe saber si los datos vinieron de HTTP o de archivos.

4. Problemas de estructuras de datos a resolver

Cada subsección describe un problema y las propiedades exigidas. No prescribe la estructura que debe utilizarse.

4.1 Qué ocurre a continuación

En cada paso del bucle principal se debe obtener el evento pendiente con menor tiempo_siguiente y, en caso de empate, con menor número de secuencia.

- Obtener el siguiente evento no puede requerir examinar todos los eventos programados.
- En las pruebas puede haber varios cientos de eventos simultáneos y esta operación se ejecutará decenas de miles de veces.
- Cuando un actor muere, sus eventos futuros no deben ejecutarse.
- Un efecto puede cancelarse antes de activarse.
- Un cambio de velocidad puede modificar el tiempo de una acción ya programada (§2.3).
- El orden debe permanecer determinista aun con cancelaciones, reprogramaciones y empates.

4.2 Traer el contenido antes de que haga falta

Cuando el jugador intenta entrar a una sala, el contenido de esa sala debe estar disponible antes de confirmar el movimiento. La estrategia normal debe evitar una espera de ~400 ms en cada puerta mediante solicitudes anticipadas por lotes.

- Se debe garantizar que el jugador nunca quede ubicado en una sala cuyo contenido aún no se conoce.
- Contenido acepta hasta 10 salas y el costo de red es prácticamente el mismo con 1 que con 10.
- El esqueleto completo está disponible, por lo que el programa puede conocer la vecindad y cercanía de las salas.
- Una sala ya solicitada no debe volver a solicitarse mientras su versión siga vigente.
- Puede existir una respuesta pendiente cuando el jugador solicita movimiento. Debe manejarse sin caída y sin que el tiempo real de red altere el reloj virtual.
- Las respuestas concurrentes o pendientes deben incorporarse en un orden determinista.

4.3 Resolución de referencias del catálogo

- Antes de mostrar o simular una entidad, su id de tipo debe resolverse a una ficha de catálogo.
- Pedir cada id por separado no cumple razonablemente el presupuesto de red.
- Los ids se descubren en orden no controlado, se repiten entre salas y pueden reutilizarse entre criptas.

- Un mismo id vigente no debe solicitarse dos veces si ya está disponible en memoria o en almacenamiento persistente.

4.4 Reutilización y caché persistente

- Las entidades del catálogo en memoria tienen un tope configurable mediante --cache-size; el valor por defecto es 25 fichas.
- Al alcanzar el tope, alguna ficha debe **liberarse de memoria**: el programa deja de mantener esa ficha accesible y volver a necesitarla obliga a recuperarla. Deben justificar cuál se libera y por qué.
- Una ficha liberada de memoria **permanece en el almacenamiento persistente**. Recuperarla cuesta una lectura de disco, no una solicitud de red. Solo genera solicitud de red una ficha que nunca se persistió o cuya versión de catálogo ya no coincide.
- **Las fichas en uso no pueden liberarse.** Una ficha referenciada por al menos una instancia activa —un enemigo vivo, un objeto en el inventario o en el suelo de la sala actual, una trampa de la sala actual— queda fuera de la política de desalojo mientras dure esa referencia. La política opera únicamente sobre las fichas que no están en uso.
- Averiguar si una ficha está en caché es una de las operaciones más frecuentes y su costo importa.
- La caché debe sobrevivir al cierre del programa.
- La versión de la cripta y la versión del catálogo deben usarse para validar lo persistido sin descargarlo todo nuevamente.
- Esqueleto y contenido de una cripta también pueden persistirse mientras coincida la versión de la cripta.

4.5 Inventario

- Debe poder recorrerse hacia adelante y hacia atrás desde la posición actual.
- Debe poder soltarse el objeto que se está viendo, en cualquier posición, sin que el costo dependa de la posición dentro del inventario.
- Equipar un objeto lo mueve al frente del orden real.
- Debe poder mostrarse una vista ordenada por peso, valor o nombre sin alterar el orden real.
- Cuando alcanza inventario_max, recoger queda prohibido hasta liberar espacio.

4.6 Pergaminos de retroceso e historial reversible

El estado debe poder volver a como estaba antes de la última acción del jugador, incluyendo todos los eventos ejecutados después de ella y antes de la siguiente decisión del jugador (§2.11).

- Está prohibido guardar copias completas del estado del juego.
- Debe almacenarse únicamente la información necesaria para invertir los cambios.
- El historial permite como máximo 5 acciones del jugador y descarta lo más antiguo al superar el límite.
- Deshacer debe restaurar reloj, agenda de eventos, rastro, vida, inventario, objetos, puertas, trampas y demás cambios pertinentes.
- El pergamino usado debe quedar consumido después de completar el retroceso.
- Los pergaminos de retroceso están excluidos del estado reversible: su cantidad no se registra en el historial ni se restaura al deshacer (§2.11).

4.7 El rastro del jugador

- La decisión de un rastreador debe costar proporcionalmente al número de salidas de su sala, no al tamaño de la cripta ni a la longitud del recorrido del jugador.
- Un rastro de 400 unidades o más de antigüedad no sirve. Decidan si lo eliminan activamente o si detectan su caducidad al consultarlo, y justifiquen.

- Con varios rastreadores, el trabajo total por ciclo no debe crecer más que linealmente con la cantidad de enemigos que realmente actúan.

4.8 Bitácora en pantalla

- La interfaz muestra únicamente los últimos 20 mensajes del juego.
- El consumo de memoria de esta bitácora debe permanecer constante durante toda la partida.
- Agregar un mensaje no debe implicar desplazar físicamente los 19 anteriores.

4.9 Ordenamiento

El programa necesita ordenar en distintos contextos: tabla de puntajes, vistas del inventario, índices del archivo de guardado y listas auxiliares del catálogo.

- Deben implementar al menos dos algoritmos de ordenamiento con características distintas.
- El programa debe escoger automáticamente cuál utilizar de acuerdo con propiedades medibles de los datos.
- La decisión debe sustentarse con mediciones propias. Consideren tamaños pequeños, datos casi ordenados y comparaciones costosas.
- No se permite list.sort() ni sorted() para los ordenamientos evaluados en esta sección.

4.10 Guardado binario de la partida

- El estado se guarda en un archivo binario de formato propio.
- Al cargar debe poder recuperarse una sala específica sin leer el archivo completo.
- El archivo debe detectar incompatibilidad con una versión distinta del formato del programa.
- Deben comparar tamaño en disco y tiempo de carga contra guardar el mismo estado en JSON.

4.11 Bitácora de partida y reproducción

Cada acción del jugador se registra en un archivo de texto de solo anexado (append). El encabezado contiene, como mínimo, id de cripta, versión de cripta, versión de catálogo y semilla. La bandera --replay partida.log reproduce la partida completa sin requerir interacción del usuario.

Si la versión indicada por un log ya no está disponible y no existe una copia local compatible, el programa debe reportar que no puede garantizar una reproducción fiel. No debe reproducir silenciosamente contra datos distintos.

Evaluación. La reproducción es obligatoria y será el mecanismo principal para reportar y repetir errores durante la evaluación.

5. Requisitos de rendimiento

5.1 Presupuesto de red por cripta

Cada cripta declara presupuesto_solicitudes en sus datos generales. Ese valor es el máximo de solicitudes HTTP que el programa puede realizar durante la partida. El presupuesto no se calcula mediante una constante o fórmula codificada en el cliente.

El servidor puede asignar presupuestos diferentes a criptas pequeñas (por ejemplo, 20 salas), medianas o grandes. Los estudiantes deben diseñar el cliente para utilizar lotes, precarga y persistencia en lugar de depender de un número específico.

- Toda solicitud HTTP cuenta, incluyendo lista de criptas, datos generales, páginas, contenido, catálogo y verificaciones de versión.
- Un reintento posterior a HTTP 429 también cuenta como solicitud realizada.
- Lecturas desde almacenamiento local o modo --offline no cuentan como solicitudes de red.
- Al finalizar la partida el programa reporta solicitudes realizadas, presupuesto_solicitudes, aciertos de caché y fallos de caché.
- La meta no es “gastar el presupuesto”: una caché caliente debería permitir usar menos solicitudes.

5.2 Presupuesto de procesamiento

Con la cripta de referencia ya cargada y excluyendo tiempo de red y renderizado, resolver un ciclo completo de eventos entre dos decisiones consecutivas del jugador debe tomar menos de 20 ms en el equipo local.

5.3 Modo de medición

La bandera --bench ejecuta cargas sintéticas y reporta tiempos por subsistema. Como mínimo debe permitir medir agenda de eventos, búsquedas de caché, recuperación de fichas liberadas, inventario, historial de retroceso, ordenamientos y operaciones de guardado/carga que aparezcan en el Documento de Decisiones.

6. Arquitectura requerida

Se exige separación Modelo–Vista–Controlador (MVC):

Componente	Responsabilidad
Modelo	Reglas del juego, entidades y estructuras de datos. No conoce HTTP, archivos ni la interfaz concreta.
Vista	Presenta el estado y solicita/recoge interacción. Puede ser de consola o gráfica.
Controlador	Traduce las acciones del usuario en operaciones del modelo y coordina el flujo de la aplicación.
Adaptadores	Acceso a red y a disco detrás de interfaces. --offline sustituye la fuente de datos sin propagar if por todo el modelo.

Los modos --replay y --bench deben ejecutarse sin instanciar la vista interactiva. Si el modelo necesita una ventana, widgets o entrada de teclado para funcionar, la separación no cumple el requisito.

La interfaz puede ser de consola o gráfica (por ejemplo con Pygame o Arcade), pero no existe bonificación por una interfaz gráfica. La evaluación se concentra en el modelo, las estructuras de datos, la evidencia y la defensa.

7. Restricciones de implementación

Deben implementar ustedes las estructuras de datos utilizadas para resolver los problemas de §4. Está prohibido sustituirlas por implementaciones equivalentes de bibliotecas.

- Los diccionarios producidos por respuesta.json() pueden usarse para leer JSON.
- Está prohibido usar dict como índice, caché, tabla de búsqueda o estructura propia para resolver los problemas evaluados.

- Sí se permite list como almacenamiento contiguo de bajo nivel, además de struct, json, requests y bibliotecas de interfaz.

Si una biblioteca permitida utiliza internamente una estructura prohibida para asuntos ajenos al problema evaluado (por ejemplo, requests para HTTP), eso no sustituye la estructura que ustedes deben implementar.

8. Documento de Decisiones de Diseño

Debe contener una sección por cada problema de §4 resuelto.

1. Problema: explíquenlo en sus palabras e indiquen qué operaciones ejecuta realmente su programa y con qué frecuencia medida.
2. Alternativas consideradas: al menos dos opciones y la razón concreta por la que una fue descartada.
3. Decisión: estructura y estrategia implementadas.
4. Análisis de complejidad: enfocado en las operaciones que el juego ejecuta con mayor frecuencia.
5. Evidencia empírica: mediciones propias obtenidas con --bench. Al menos una sección debe medir directamente la alternativa descartada.
6. Límites conocidos: condiciones bajo las cuales la decisión dejaría de ser apropiada.

Criterio. Una sección que solo diga “usamos X porque es O(log n)” no demuestra el razonamiento requerido. Se evalúa la relación entre operaciones reales, restricciones y evidencia.

Concluir que una alternativa sencilla es suficiente para cierta escala es válido si las mediciones lo sustentan y el programa sigue cumpliendo los requisitos cuando la cripta crece.

9. Defensa y revisión del proyecto

Durante la revisión del programa se apuntará a una cripta que los grupos no habrán visto previamente. Puede tener más salas, actores, tipos, objetos y efectos simultáneos que las criptas usadas durante el desarrollo.

- No puede haber números de salas, ids de entidades, cantidad de páginas, presupuestos ni estadísticas de una cripta específica escritos en el código.
- Cada integrante debe poder explicar cualquier decisión del Documento de Decisiones, no solo el componente que implementó personalmente.
- Se verificará el presupuesto_solicitudes de la cripta utilizada y el requisito de procesamiento de §5.2.
- Se utilizará --replay para reproducir escenarios y --bench para contrastar afirmaciones de rendimiento.

10. Entregas y documentación obligatoria

El proyecto debe estar alojado en Github, de manera que cada grupo debe crear un repositorio para tal fin.

10.1 README

Cada entrega debe incluir un README actualizado con instrucciones suficientes para ejecutar el proyecto desde un entorno limpio: versión de Python, dependencias, comando de instalación, comandos principales, modo --offline, --replay y --bench según estén disponibles, y ubicación de los archivos generados.

10.2 Interfaz funcional mínima

La sintaxis exacta de comandos puede variar, pero la interfaz final debe permitir como mínimo:

- seleccionar una cripta e iniciar una partida;
- moverse en las cuatro direcciones disponibles;

- abrir una puerta cuando corresponda;
- atacar un enemigo seleccionado;
- esperar;
- recoger, usar, equipar y soltar objetos;
- recorrer y consultar el inventario, incluyendo vistas ordenadas;
- usar un pergamino de retroceso;
- guardar y cargar la partida;
- terminar la sesión de forma controlada.

La vista debe mostrar suficiente información para tomar esas decisiones: sala actual, salidas, enemigos, objetos del suelo, vida y estadísticas relevantes, estado de efectos y los últimos 20 mensajes.

10.3 Uso de modelos de lenguaje (LLM)

El uso de un LLM está permitido como herramienta de apoyo para comprender conceptos, explorar alternativas, revisar código, generar casos de prueba, mejorar documentación o proponer implementaciones. El código o texto generado con ayuda de un LLM se considera responsabilidad del grupo exactamente igual que cualquier otro código entregado.

Cada entrega debe incluir un archivo de bitácora, por ejemplo PROMPTS_LLM.md, con todos los prompts relacionados con el proyecto que el equipo haya enviado a un LLM desde la entrega anterior. Por cada entrada debe indicarse:

- fecha aproximada;
- herramienta o modelo utilizado, si se conoce;
- prompt enviado, de forma textual;
- una frase breve indicando para qué se utilizó la respuesta (por ejemplo: “comparar alternativas de caché”, “revisar un caso borde del replay”).

No es necesario copiar las respuestas completas del LLM. Si no se utilizó ningún LLM durante el periodo, la bitácora debe indicarlo explícitamente. No incluyan credenciales, datos personales ni información ajena al proyecto.

Responsabilidad académica. La bitácora no reduce la exigencia de la defensa. Todo integrante debe poder explicar y modificar el código entregado, independientemente de que haya sido escrito manualmente o con asistencia de un LLM.

11. Evaluación

Rubro	Peso dentro del proyecto
Cliente y presupuesto de red	25 %
Juego funcional	20 %
Caché, persistencia, retroceso y mediciones	25 %
Documento de Decisiones de Diseño	15 %

Rubro	Peso dentro del proyecto
Rendimiento	15 %

12. Preguntas para arrancar

No deben responderse por escrito. Sirven para orientar la primera discusión técnica del grupo:

- Con varios cientos de eventos programados, ¿qué operación se ejecutará decenas de miles de veces y cuánto cuesta hacerla mal?
- El jugador está en una sala con tres salidas y todavía no decide. ¿Qué contenido conviene tener listo antes de que elija?
- La consulta de contenido acepta 10 salas y cuesta casi lo mismo con una que con veinte. ¿Cómo decidirán qué salas incluir en cada lote?
- La caché recibirá muchas más búsquedas que inserciones. ¿Cómo afecta eso la estructura elegida?
- ¿Qué información mínima hay que registrar para revertir un ataque? ¿Una recolección? ¿Una acción enemiga? ¿Un cambio de velocidad? ¿Un evento de puerta?
- ¿Cómo evitarán que el orden en que llegan dos respuestas HTTP cambie la partida reproducida?
- ¿Qué parte del programa se degrada primero si la cripta crece 50 veces?

Apéndice A. Resumen del contrato de datos

Dato	Origen	Persistencia/validez
Lista de criptas	GET /criptas	Puede refrescarse al iniciar; no contiene el estado del juego.
Datos generales	GET /criptas/<id>	Válidos para la version de la cripta.
Esqueleto	GET /criptas/<id>/salas?pagina=n	Válido mientras coincida la version de la cripta.
Contenido de salas	GET /criptas/<id>/contenido?salas=...	Válido mientras coincida la version de la cripta.
Ficha de catálogo	GET /catalogo?ids=...	Válida mientras coincida la version global del catálogo.
Versión de cripta	GET /criptas/<id>/version	Consulta liviana para validar datos persistidos.

Dato	Origen	Persistencia/validez
Versión de catálogo	GET /catalogo/version	Consulta liviana para validar fichas persistidas.

Apéndice B. Casos que deben tener un resultado definido

- El jugador intenta recoger con el inventario lleno: la acción no se ejecuta ni consume tiempo; debe soltar primero.
- El jugador intenta abrir una puerta sin la llave correcta: la acción se rechaza y no consume tiempo.
- Un errante no tiene salidas abiertas: espera con costo 100.
- El rastro de la sala actual se debe refrescar por cada acción del jugador.
- Un rastreador tiene dos vecinos con rastro igualmente fresco: elige menor id de sala.
- Un enemigo muere con eventos futuros programados: esos eventos no producen acciones.
- Un antídoto cancela veneno antes del siguiente pulso: los pulsos futuros cancelados no hacen daño.
- Una puerta se cierra automáticamente mientras un actor ya está en la sala del otro lado: el actor permanece allí; solo se impide atravesar de nuevo hasta abrir.
- La vida regenerada nunca supera vida_max.
- Una poción de curación nunca eleva vida por encima de vida_max.
- Un movimiento hacia contenido aún pendiente espera la información antes de confirmar la entrada; la espera real no mueve el reloj virtual.
- La carga de un replay con versión incompatible debe reportar la incompatibilidad en vez de continuar silenciosamente.
- El jugador usa un pergamino cuando el intervalo a revertir incluía la recolección de otro pergamino: la reversión no devuelve ese pergamino al suelo y la cantidad final refleja únicamente el consumo de la unidad usada.
- El jugador usa dos pergaminos seguidos: como usarlos no consume tiempo virtual ni se registra como acción del jugador, el segundo retrocede la acción anterior a la que ya se revirtió.